

What's More in Tracing with LTTng

The next big step is to trace userspace applications and understand how to view the overwhelming trace information easily using some GUI tools. Read on to dive deeper into tracing.



In the article that appeared in the March 2014 issue of *OSFY*, I introduced tracing and, more specifically, LTTng. I included a small section towards the end in which you could generate a small kernel trace. Now, it's time to have a look at more advanced use cases. In this article, I will introduce LTTng for tracing applications in userspace. Eventually, we'll discuss how to use the power of the Eclipse Tracing and Monitoring Framework (TMF) and other tools to gather information from combined userspace and kernel traces, using a real life example.

Userspace tracing with LTTng

If you remember, we enabled tracepoint in the kernel the last time, and gathered data and information about this. What if

you wanted to trace specific 'points' in your application at the userspace level, at the same blazingly fast speed! As an example, consider that you want to know how much time a `handle_newconnect()` call takes in your Web server and how it interacts with the system. Or maybe you wish to know why the APT (package manager for Debian/Ubuntu) takes so much time installing your package? (*Hint*: It blocks for about 37 per cent of total elapsed time. But why, and where?)

Before we find the answers to all those questions, let us look at the `thttpd` Web server. Let us first make it ready for LTTng/UST-the userspace tracing library of LTTng. The first step is inserting a tracepoint function call in a specific function in the `thttpd` source. This is known as *instrumenting* the application. For that we need something called the *tracepoint provider*.